

SESIÓN 12 JWT en Spring Security (Stateless)

"Autenticación sin sesión: el token viaja en cada request."

Bloque 1 — ¿Por qué JWT ahora?

Con HTTP Basic vs Con JWT

Con HTTP Basic:

- en cada request envías usuario+password (aunque sea codificado en base64)
- no es ideal para frontends modernos
- no es "stateless" real en APIs de producción

Con JWT:

- el usuario hace login 1 vez
- recibe un token
- a partir de ahí manda: Authorization: Bearer



Mensaje clave:

"El servidor no guarda sesión: el token prueba tu identidad."

Bloque 2 — Conceptos esenciales JWT

¿Qué es JWT?

JSON Web Token: un token con formato estándar, compacto, firmado.

Un JWT típico tiene 3 partes:

HEADER.PAYLOAD.SIGNATURE

Header

algoritmo de firma, tipo

Payload

claims (datos), ej: usuario, roles, expiración

Signature

firma que evita manipulación

¿JWT está cifrado?

No. Por defecto está firmado, no cifrado.

Cualquiera puede leer el payload si tiene el token.

Lo importante es que no pueden modificarlo sin romper la firma.

Stateless

Stateless significa:



el backend NO guarda sesión



cada request trae el token



el backend valida firma + expiración



401 vs 403 en JWT

401

token no existe / inválido / expirado

403

token válido, pero no tienes permisos



Bloque 3 — Dependencias (Maven)

pom.xml (JWT con JJWT)

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

Spring Security ya lo tienes: `spring-boot-starter-security`.



Bloque 4 — Configuración en properties

application.properties

```
# OJO: en producción esto debe ir en variables de entorno, no hardcodeado
jwt.secret=CHANGE_ME_TO_A_LONG_RANDOM_SECRET_KEY_32+CHARS
jwt.expiration-ms=3600000
```



Bloque 5 — Estructura del proyecto

```
com.cladsb1.books
├── config
│   └── SecurityConfig
└── security
```



Bloque 6 — JWT Service (crear/validar token)

security/JwtService.java

```
package com.cladsb1.books.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import java.nio.charset.StandardCharsets;
import java.security.Key;
import java.util.Date;
import java.util.List;

/**
 * Responsabilidad:
 * - Generar tokens JWT
 * - Extraer usuario/claims
 * - Validar firma y expiración
 * Importante:
 * - JWT se firma con una clave secreta (HMAC)
 * - No se cifra (el payload se puede leer)
 */
@Service
public class JwtService {

    private final Key signingKey;
    private final long expirationMs;

    public JwtService(
        @Value("${jwt.secret}") String secret,
        @Value("${jwt.expiration-ms}") long expirationMs
    ) {
        // Generamos una Key a partir del secret.
        // Para HS256 conviene un secret largo (>= 32 chars).
        this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMs = expirationMs;
    }

    /**
     * Genera un token JWT con:
     * - subject: username
     * - claim "roles": lista de roles
     * - issuedAt / expiration
     */
    public String generateToken(UserDetails user) {
        Date now = new Date();
        Date expiry = new Date(now.getTime() + expirationMs);

        List<String> roles = user.getAuthorities()
            .stream()
            .map(GrantedAuthority::getAuthority) // ej: "ROLE_ADMIN"
            .toList();

        return Jwts.builder()
            .setSubject(user.getUsername())
            .claim("roles", roles)
            .setIssuedAt(now)
            .setExpiration(expiry)
            .signWith(signingKey, SignatureAlgorithm.HS256)
            .compact();
    }

    /**
     * Extrae el username (subject) desde el token.
     * Si el token es inválido, lanzará excepción.
     */
    public String extractUsername(String token) {
        return parseClaims(token).getBody().getSubject();
    }

    /**
     * Valida:
     * - firma correcta
     * - token no expirado
     * - estructura correcta
     * Si algo falla, devuelve false.
     */
    public boolean isValid(String token) {
        try {
            Claims claims = parseClaims(token).getBody();
            return claims.getExpiration().after(new Date());
        } catch (JwtException | IllegalArgumentException ex) {
            return false;
        }
    }

    private Jws<Claims> parseClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(signingKey)
            .build()
            .parseClaimsJws(token);
    }
}
```



Punto didáctico importante:

En generateToken() usamos streams para mapear roles. Se puede cambiar por un for (anexo al final).



Bloque 7 — Filter: leer Bearer token y autenticar



security/JwtAuthenticationFilter.java

```
package com.cladsb1.books.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import org.springframework.http.HttpHeaders;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

/**
 * Filtro JWT:
 * - Se ejecuta una vez por request
 * - Si hay header Authorization: Bearer
 *   valida el token, carga el usuario y lo mete en el SecurityContext.
 * - Si no hay token, deja continuar (request anónima),
 *   y las reglas de seguridad decidirán si se permite o no.
 */
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService,
                                   UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);

        // 1) Si no hay Authorization o no empieza por Bearer, no autenticamos aquí.
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        // 2) Extraemos token
        String token = authHeader.substring(7);

        // 3) Validamos token (firma + expiración)
        if (!jwtService.isTokenValid(token)) {
            // No seteamos autenticación -> acabará en 401 si endpoint requiere auth
            filterChain.doFilter(request, response);
            return;
        }

        // 4) Extraemos username y cargamos el usuario
        String username = jwtService.extractUsername(token);

        // 5) Si aún no hay autenticación en el contexto, la creamos
        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails user = userDetailsService.loadUserByUsername(username);

            // Creamos Authentication con authorities (roles)
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(
                    user,
                    null,
                    user.getAuthorities()
                );
            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

            // 6) Guardamos la autenticación en el contexto
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }

        // 7) Continuamos con la cadena de filtros
        filterChain.doFilter(request, response);
    }
}
```



Mensaje clave:

"El JWT filter traduce un Bearer token a un usuario autenticado dentro de Spring."



Bloque 8 — DTOs de autenticación

 security/AuthDtos.java

```
package com.cladsb1.books.security;

import jakarta.validation.constraints.NotBlank;

/**
 * DTOs para AuthController.
 */
public class AuthDtos {

    public record LoginRequest(
        @NotBlank String username,
        @NotBlank String password
    ) {}

    public record LoginResponse(
        String token
    ) {}
}
```



Bloque 9 — AuthController (login → token)

 controller/AuthController.java

```
package com.cladsb1.books.controller;

import com.cladsb1.books.security.AuthDtos.LoginRequest;
import com.cladsb1.books.security.AuthDtos.LoginResponse;
import com.cladsb1.books.security.JwtService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.*;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.*;

/**
 * Endpoint de login:
 * - Recibe username/password
 * - Usa AuthenticationManager para validar credenciales
 * - Devuelve JWT
 */
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;

    public AuthController(AuthenticationManager authenticationManager,
        JwtService jwtService) {
        this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest request) {

        // 1) Pedimos a Spring que autentique al usuario
        Authentication auth = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.username(),
                request.password()
            )
        );

        // 2) Si llegamos aquí, las credenciales son correctas.
        // auth.getPrincipal() suele ser UserDetails
        UserDetails user = (UserDetails) auth.getPrincipal();

        // 3) Generamos token
        String token = jwtService.generateToken(user);

        return ResponseEntity.ok(new LoginResponse(token));
    }
}
```




Bloque 10 — SecurityConfig JWT (stateless)



config/SecurityConfig.java

```
package com.cladsb1.books.config;

import com.cladsb1.books.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

/**
 * Configuración Security:
 * - Stateless (sin sesión)
 * - JWT Filter antes del filtro estándar de username/password
 * - Rutas públicas: swagger + login + (opcional search)
 */
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable()) // Muy importante: API stateless
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                // Login público
                .requestMatchers("/api/auth/**").permitAll()
                // Puedes decidir si search es público o no
                .requestMatchers("/api/books/search").permitAll()
                // Books protegido: solo ADMIN (ejemplo de reglas por endpoint)
                .requestMatchers("/api/books/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            );

        // Insertamos nuestro filtro JWT antes del filtro estándar
        http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    /**
     * AuthenticationManager se usa en el login endpoint para autenticar credenciales.
     */
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
        throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    /**
     * Demo con usuarios en memoria (como sesión 11).
     * En proyectos reales, esto suele venir de BD.
     */
    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
        UserDetails admin = User.builder()
            .username("admin")
            .password(encoder.encode("admin123"))
            .roles("ADMIN")
            .build();

        UserDetails user = User.builder()
            .username("user")
            .password(encoder.encode("user123"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }
}
```




Bloque 11 — Cómo probarlo en Swagger

01

Login

POST /api/auth/login con:

```
{
  "username": "admin",
  "password": "admin123"
}
```

02

Copia el token

03

En Swagger: Authorize → pega:

Bearer <token>

04

Prueba:

- GET /api/books/search (si lo dejaste público) → 200
- POST /api/books → 201 si admin
- Con user/user123 → 403



Bloque 12 — Errores típicos

✗ No poner STATELESS → comportamientos raros con sesión

✗ No añadir el filtro addFilterBefore → nunca autentica con JWT

✗ Secret corto → problemas de firma HS256

✗ Pensar que JWT cifra datos → no, solo firma

✗ Meter datos sensibles en el payload



Anexo: versión SIN streams del mapping de roles

En JwtService.generateToken():

```
List<String> roles = new ArrayList<>();
for (GrantedAuthority ga : user.getAuthorities()) {
  roles.add(ga.getAuthority());
}
```